

# Embedded AI on FPGA using High Level Synthesis

Cougouluegne Thomas

4 Janvier 2026

## Table des matières

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                | <b>2</b> |
| 1.1      | Environnement . . . . .                            | 2        |
| 1.2      | Méthodologie et développement . . . . .            | 2        |
| 1.3      | Versions implémentées . . . . .                    | 3        |
| 1.3.1    | Version logicielle (SOFT) . . . . .                | 3        |
| 1.3.2    | Version matérielle séquentielle (HW_SEQ) . . . . . | 3        |
| 1.3.3    | Version finale (HW_PAR) . . . . .                  | 3        |
| <b>2</b> | <b>Validation et analyse des performances</b>      | <b>3</b> |
| 2.1      | Performances de la version logicielle . . . . .    | 3        |
| 2.2      | Performances de la version HW_SEQ . . . . .        | 3        |
| 2.3      | Utilisation des ressources FPGA . . . . .          | 3        |
| 2.4      | Résultats . . . . .                                | 4        |
| 2.4.1    | Vivado HLS . . . . .                               | 4        |
| 2.4.2    | SDSoC . . . . .                                    | 4        |
| <b>3</b> | <b>Conclusion</b>                                  | <b>4</b> |

# 1 Introduction

L'objectif de notre projet est de concevoir et d'implémenter un accélérateur de réseau de neurones convolutif (CNN) sur un système CPU/FPGA en utilisant la synthèse de haut niveau (HLS). Le CNN est basé sur l'architecture LeNet-5, l'une des premières et des plus célèbres architectures dédiées à la reconnaissance de chiffres manuscrits, et sera exécuté sur une carte ZedBoard équipée d'un SoC Xilinx Zynq-7000. Les cœurs ARM Cortex-A9 sont chargés du contrôle, de la gestion du système et de l'exécution de la version logicielle de référence en virgule flottante. La logique programmable du FPGA est quant à elle utilisée pour l'accélération matérielle du réseau de neurones, implémenté exclusivement en arithmétique fixe. L'objectif est d'atteindre une inférence en temps réel sur le jeu de données MNIST, avec des améliorations en termes de vitesse d'exécution et d'efficacité énergétique par rapport à une implémentation purement logicielle exécutée sur le processeur ARM.

Trois versions seront comparées :

- **SOFT** : version logicielle exécutée sur processeur ARM,
- **HW\_SEQ** : accélérateur matériel généré par Vivado HLS sans optimisation manuelle,
- **HW\_PAR** : non implémentée dans ce projet (discussion théorique).

L'analyse porte sur le temps de traitement, le nombre de cycles et l'utilisation des ressources FPGA.

## 1.1 Environnement

Le projet est réalisé sur une plateforme SoC Xilinx Zynq intégrant :

- - un processeur ARM Cortex-A9 sous Linux,
- - une logique programmable FPGA,
- - les outils Vivado HLS et SDSoC.

Les performances seront mesurées à l'aide des compteurs `sds_clock_counter` et de la fonction Linux `gettimeofday`.

## 1.2 Méthodologie et développement

Le développement du projet s'est déroulé en plusieurs étapes clairement distinctes entre la partie logicielle et la partie matérielle.

Dans un premier temps, une version logicielle de référence du réseau LeNet a été utilisée en virgule flottante (`float`) sur le processeur ARM. Cette version sert uniquement de référence fonctionnelle afin de valider la justesse des calculs et de comparer les résultats avec l'implémentation matérielle.

Dans un second temps, l'intégralité du réseau de neurones est réimplémentée en arithmétique fixe pour l'accélération sur FPGA. Ce choix est imposé par l'architecture matérielle du FPGA, qui ne dispose pas d'unité de calcul en virgule flottante matérielle et pour lequel une implémentation fixe est beaucoup plus efficace en termes de ressources et de performances.

Chaque couche du réseau (convolution, pooling, couches fully connected et softmax) a été développée en respectant strictement les contraintes de la synthèse HLS :

- - absence d'allocation dynamique.
- - utilisation de tableaux statiques.
- - boucles déterministes et bornées.

La validité fonctionnelle de l'implémentation fixe a été vérifiée en comparant les sorties avec la version flottante de référence exécutée sur le processeur ARM. (Acc = 97.3%)

## 1.3 Versions implémentées

### 1.3.1 Version logicielle (SOFT)

La version logicielle exécute l'intégralité du réseau sur le processeur ARM. Elle sert de référence fonctionnelle et temporelle.

### 1.3.2 Version matérielle séquentielle (HW\_SEQ)

La version HW\_SEQ correspond à un portage matériel direct et non optimisé du code C, sans utilisation de directives HLS. Cette version n'est pas optimisée et ne présente aucun pragma. Elle serait donc très mal optimisée et bien moins efficace que les deux autres.

### 1.3.3 Version finale (HW\_PAR)

Version avec optimisation manuelle appliquées (pipeline, unrolling, partitionnement mémoire). Permet d'accélérer le système et de mieux exploiter le FPGA.

Les outils de synthèse de haut niveau, tels que Vivado HLS, permettent de générer automatiquement une architecture matérielle à partir d'un code C/C++. Cependant, sans indication explicite du concepteur, l'outil adopte une approche conservatrice et génère généralement une architecture majoritairement séquentielle.

Les **directives HLS**, également appelées **pragmas**, permettent de guider le processus de synthèse afin d'exploiter efficacement le parallélisme offert par le FPGA. Elles n'affectent pas la sémantique fonctionnelle du code C, mais influencent directement l'architecture matérielle.

La directive `#pragma HLS PIPELINE` permet de transformer une boucle séquentielle en une architecture pipeline. Une nouvelle itération de la boucle peut ainsi être lancée avant la fin de la précédente, ce qui réduit l'*Initiation Interval* (II). Cette optimisation est particulièrement efficace pour les calculs répétitifs de convolutions, où les mêmes opérations sont appliquées à de nombreuses données successives.

La directive `#pragma HLS UNROLL` permet de déplier une boucle, en exécutant plusieurs itérations en parallèle. Cela augmente le parallélisme spatial au prix d'une consommation plus élevée de ressources matérielles (DSP, LUT). On l'utilisera donc uniquement sur des petites boucles récurrentes.

## 2 Validation et analyse des performances

### 2.1 Performances de la version logicielle

La version logicielle présente un temps de traitement élevé en raison de l'exécution séquentielle des convolutions et des couches fully-connected. Elle n'est absolument pas optimisée. (Acc = 97.3%)

### 2.2 Performances de la version HW\_SEQ

### 2.3 Utilisation des ressources FPGA

Les ressources analysées sont :

- 1. BRAM pour le stockage.
- 2. DSP pour les opérations arithmétiques.
- 3. LUT pour la logique de contrôle.

## 2.4 Résultats

### 2.4.1 Vivado HLS

| Version           | Cycles    | BRAM | DSP | LUT |
|-------------------|-----------|------|-----|-----|
| HW_SEQ (baseline) | 8 715 887 | 18%  | 5%  | 5%  |

TABLE 1 – Vivado HLS

L'ajout de directives HLS (pragmas) permettra d'exploiter le parallélisme du FPGA et de réduire fortement le nombre de cycles d'exécution par rapport à la version HW\_SEQ.

### 2.4.2 SDSoC

| Version                | Tglobal | Thw_min     | Thw_max     | Thw_avg     | Thw_avg_s |
|------------------------|---------|-------------|-------------|-------------|-----------|
| SW (Cortex-A9 667 MHz) | 617 s   | 12 870 000  | 27 000 000  | 12 910 000  | 19.4 ms   |
| HW_SEQ                 | 2734 s  | 182 200 000 | 196 500 000 | 182 311 605 | 273.3 ms  |

TABLE 2 – Performances SDSoC

Ce tableau permet de comparer les performances globales du système pour les différentes versions, en mettant en évidence les différences de performance entre les versions software et matérielle. La version accélérée si implémentée devrait d'ailleurs être bien plus efficace que les deux présentes ici.

## 3 Conclusion

Ce projet a permis de mettre en œuvre un réseau de neurones convolutif sur FPGA à l'aide d'outils HLS. Les résultats obtenus illustrent les bénéfices et les limites d'une approche HLS sans optimisation manuelle. Des améliorations significatives devraient être observées après l'ajout de parallélisme via pragmas. Qui permettraient de profiter au maximum des optimisations disponibles sur la plateforme FPGA.